

Dynamics Test Procedure

5/7/2026

Test procedure name:	Dynamics Test Procedure
Team name:	Dynamics Team
Sub-system:	Dynamics
Code Version:	DynamicsV4.m
Simulation Platform:	MATLAB
Prepared By:	Geenadie Rathnayake
Date Conducted:	5/17/2026
Conducted By:	Kyle Wittcoff
Document ID:	

Test Result	PASS / FAIL
DNR-1	PASS / FAIL
DNR-2	PASS / FAIL
DNR-3	PASS / FAIL
DNR-4	PASS / FAIL
DNR-5	PASS / FAIL
DNR-6	PASS / FAIL
DNR-7	PASS / FAIL
DNR-8	PASS / FAIL

Contents

1	Objective	2
2	Scope of Test	2
1	System Being Tested	2
2	Type of Test	3
3	Required Measurements	3
3	Dependence on Other Test Procedures	3
4	Equipment Required	3
5	Procedure	4
1	Setup	4
2	Software Initialization	4
3	Initial State Vector Assembly	4
4	Control Vector Assembly	5
5	Dynamics Integration Loop	5
6	Post Processing	5
7	Test Evaluation	6
8	Shut Down	6

1 Objective

The objective of this test plan is to verify and validate the dynamics model of the Sealglider by comparing the simulated outputs to real nominal mission data. The dynamics code will take in parameters from the glider data files and then propagate the glider state over each time interval. These simulated outputs will be compared to the mission data either recorded by onboard sensors or the estimation outputted by the glider from its Hydrodynamic Model.

The outputs evaluated in this test are depth, vertical velocity, pitch, roll, heading, pitch rate, roll rate, and heading rate. These outputs are evaluated because they are used by the Diagonoser to assess glider behavior. The test will pass if each output is modeled within the allowable percent error specified by the following dynamics requirements.

- **DNR-1:** Depth (z) of a dive must be modeled within 6.6% error of onboard recorded depth of a dive.
- **DNR-2:** Vertical velocity (w) of a dive must be modeled within 3.3% error of onboard recorded vertical speed dz/dt of a dive.
- **DNR-3:** Pitch of a dive must be modeled within 6.6% error of onboard recorded pitch angle of a dive.
- **DNR-4:** Roll of a dive must be modeled within 6.6% error of onboard recorded roll angle of a dive.
- **DNR-5:** Heading of a dive must be modeled within 6.6% error of onboard recorded heading angle of a dive.
- **DNR-6:** Pitch rate of a dive must be modeled within 3.3% error of onboard recorded pitch rate of a dive.
- **DNR-7:** Roll rate of a dive must be modeled within 3.3% error of onboard recorded roll rate of a dive.
- **DNR-8:** Heading rate of a dive must be modeled within 3.3% error of onboard recorded heading rate of a dive.

2 Scope of Test

1 System Being Tested

The system being tested is the Seaglider dynamics code, specifically the portion of the model that propagates the vehicle state based on:

- Current state vector
- Actuation/control vector
- Hydrodynamic coefficient matrices
- Vehicle parameters
- Environmental or mission-specific inputs (density, pressure, density, etc.)

This applies to all dynamics code test cases defined in the Dynamics Test Plan.

2 Type of Test

This is a model verification and validation test. The test is a verification test because the dynamics code output is checked against the requirements. The test is a validation test because the simulated outputs are compared against real mission data from past mission dives.

3 Required Measurements

The measurements that will be taken during each test case are depth, vertical velocity, pitch, roll, heading, pitch rate, roll rate, and heading rate.

3 Dependence on Other Test Procedures

List any test procedures associated with this plan.

Procedure ID	Description

4 Equipment Required

Equipment/Tools	Description	Qty	Notes	Check
Laptop	At least 8 GB RAM and 256 GB SSD is required	1		
MATLAB R2024a	Simulation environment. Add ons needed: • Aerospace Toolbox • Control System Toolbox • Optimization Toolbox • Robust Control Toolbox	1		
dynamicsV4	Latest code that models the dynamics of glider (5/17/2026 Version 4 was the latest code)	1		
ode solver	Solves dynamicsV3 function for discrete time steps given a current condition	1		
Parameter and Coefficients files for nominal case	Input dataset	1		
Mission data file	Input dataset for Test ID TDN-4 and TDN-5	1		
Test script	Matlab script to run the test cases where parameters and coefficients can be inputted and call the ode function to run the dynamics code	1		

5 Procedure

1 Setup

1. Confirm that the correct dynamics code version ((DynamicsV3) is being used.
2. Confirm that the approved test script is available.
3. Confirm that the coefficient matrices from wind tunnel testing are available.
4. Confirm that the parameter file is available.
5. Confirm that actuation inputs for each Test ID case is available (Ex: Edmonds dive 5 mission log / .nc files)
6. Confirm that all mission data fields required to initialize the dynamics code are present, including:
 - Time
 - Actuation states
 - Depth
 - Vertical velocity
 - Orientation states
7. Confirm that all data are in consistent units.
8. Confirm that the selected test case ID matches the test plan.
9. Record the selected dive number, mission phase, start time, end time, and test window length.

2 Software Initialization

1. Open MATLAB.
2. Navigate to the project directory containing the dynamics code and test script.
3. Add required folders to the MATLAB path.
4. Open the test script.
5. Load the coefficient matrices file.
6. Load the vehicle parameter file.
7. Open the mission data file, if applicable, and erify that the selected actuation states are available for every time step in the mission data file.
8. Load the mission data file into the test script.
9. Open the Test Matrix file to record simulated states.

3 Initial State Vector Assembly

1. Select the first time step from the mission data, if applicable (TDN-4, TDN-5), or set the time step to 10 seconds (TDN-1, TDN-2, TDN-3).
2. Extract the initial orientation, position and velocities of the glider from the mission data at the start of the selected window, if applicable (TDN-4, TDN-5).
3. Assemble the initial state vector, X_0 using data extracted in the last step (TDN-4, TDN-5) or use the given initial state vector (TDN-1, TDN-2, TDN-3) in the Dynamics Test Plan.
4. Record the full initial state vector used for the test in the Test Matrix.
5. Verify that X_0 contains no missing, NaN, or Inf values.

4 Control Vector Assembly

1. At the first actuation time step, extract the actuation state from the Mission Data file, if applicable or use the actuation state given in the Test Needs Matrix under Dynamics Test Plan.
2. Assemble the control vector, U , in the format required by the dynamics function.
3. Verify that the control vector contains all required actuation inputs.
4. Record the control vector for the first time step in the Test Matrix.
5. Load the Coefficients file and Parameters file into the test script.

5 Dynamics Integration Loop

For each time interval in the selected test window, perform the following steps:

1. Define the current time interval as $[t(i-1), t(i)]$ where $t(i-1)$ is the previous time and $t(i)$ is the current time.
2. Calculate the time step:

$$\Delta t = t(i) - t(i-1)$$
3. Extract the actuation states at the current time from the Mission Data file.
4. Assemble the control vector, U .
5. Pass the following into `ode` function in the test script:
 - Current time interval $[t(i-1), t(i)]$
 - Dynamics function
 - Initial state vector X_0
 - Current control vector U
 - Coefficients
 - Parameters
6. Within `ode` function, call the dynamics function.
7. Run the test script.
8. Extract the final state $\mathbf{X}$ from the `ode` outputs.
9. Record $\mathbf{X}(1)$, $\mathbf{X}(2)$, $\mathbf{X}(3)$, $\mathbf{X}(4)$, $\mathbf{X}(5)$, $\mathbf{X}(6)$, $\mathbf{X}(9)$, and $\mathbf{X}(12)$ from the simulated state vector $\mathbf{X}$ at time $t(i)$ in the Test Matrix (Note that these elements in the vector correspond to the depth, vertical velocity, orientation, and orientation rates).
10. Use the final state from the current interval as the initial condition for the next interval.
11. Repeat this process until the selected test window is complete.

6 Post Processing

1. Compile the simulated state vector outputs over the full test window.
2. Extract the corresponding real mission data over the same time window.
3. Extract the required comparison outputs:
 - Depth
 - Vertical velocity
 - Roll
 - Pitch
 - Yaw
 - Angular rates, if required by the test case

4. Align simulated outputs and mission data by time index.
5. Generate plots comparing:
 - Simulated depth vs. mission depth
 - Simulated vertical velocity vs. mission vertical velocity
 - Simulated orientation vs. mission orientation
 - Control inputs vs. time, if needed for interpretation

7 Test Evaluation

1. Compare each modeled output to the corresponding onboard recorded value at each evaluated time step.
2. Calculate the absolute difference between the modeled value and onboard recorded value for each output.
3. Compare each absolute difference against the requirement-defined allowable tolerance. For example,

$$|z_{\text{modeled}}(t) - z_{\text{recorded}}(t)| \leq 8.6 \text{ m}$$

4. Determine pass/fail for each output.
 - **Pass Criteria:** A test case passes if each modeled output remains within its requirement-defined allowable tolerance of the corresponding onboard recorded value at each evaluated time step during the dive.
 - **Fail Criteria:** A test case fails if any modeled output exceeds its requirement-defined allowable tolerance at one or more evaluated time steps during the dive.
5. Record whether each test case passes or fails each of the defined requirements.
6. If the test fails, document:
 - Which output failed
 - Maximum absolute difference
 - Time at which the maximum absolute difference occurred
 - Requirement tolerance that was exceeded
 - Possible cause, if known

8 Shut Down

1. Save all test results in the Dynamics test folder.
2. Close Matlab.